부록 · LangChain4j · Spring AI

Spring Boot Java + Kotlin XRP 멀티 타임프레임 분석

15분·60분·240분·일봉·주봉·월봉·연봉 → 지표 → 신호점수 → Gemma3

1. 전체 구조

```text
Upbit KRW-XRP
  ├─ 15분봉  → 초단기
  ├─ 60분봉  → 단기
  ├─ 240분봉 → 중단기
  ├─ 일봉     → 중기
  ├─ 주봉     → 장기
  ├─ 월봉     → 장기
  └─ 연봉     → 초장기
          ↓
각 프레임 IndicatorService
          ↓
각 프레임 SignalService
          ↓
MultiTimeframeService
          ↓
가중 종합 상승/하락 점수
          ↓
LangChain4j → Ollama → Gemma3
          ↓
초단기/단기/중기/장기 종합 설명
```

2. 시간봉과 역할 비교

프레임	용도	예제 가중치
15분	초단기 흐름	5%
60분	단기 흐름	10%
240분	중단기 흐름	15%
일봉	중기 방향	25%
주봉	장기 추세	20%
월봉	장기 추세	15%
연봉	초장기 위치	10%

가중치는 학습용 예제이며 실제 적용 전 백테스트로 조정하는 것이 좋습니다.

3. TimeframeAnalysisDto

Java

```text
public record TimeframeAnalysisDto(
    String timeframe,
    IndicatorDto indicators,
    SignalDto signal
) {}
```

Kotlin

```kotlin
data class TimeframeAnalysisDto(
    val timeframe: String,
    val indicators: IndicatorDto,
    val signal: SignalDto
)
```

4. UpbitService - Java

```java
@Service
public class UpbitService {

    private final RestClient restClient;

    public UpbitService(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.upbit.com")
            .build();
    }

    public List<CandleDto> getMinuteCandles(int minute) {
        return getCandles("/v1/candles/minutes/" + minute);
    }

    public List<CandleDto> getDayCandles() {
        return getCandles("/v1/candles/days");
    }

    public List<CandleDto> getWeekCandles() {
        return getCandles("/v1/candles/weeks");
    }

    public List<CandleDto> getMonthCandles() {
        return getCandles("/v1/candles/months");
    }

    public List<CandleDto> getYearCandles() {
        return getCandles("/v1/candles/years");
    }

    private List<CandleDto> getCandles(String path) {

        CandleDto[] result = restClient.get()
            .uri(b -> b.path(path)
                .queryParam("market", "KRW-XRP")
                .queryParam("count", 200)
                .build())
            .retrieve()
            .body(CandleDto[].class);

        if (result == null || result.length == 0) {
            throw new IllegalStateException("캔들 데이터가 없습니다: " + path);
        }

        List<CandleDto> list =
            new ArrayList<>(Arrays.asList(result));

        Collections.reverse(list);
        return list;
    }
}
```

5. UpbitService - Kotlin

```kotlin
@Service
class UpbitService(
    builder: RestClient.Builder
) {
    private val restClient =
        builder.baseUrl("https://api.upbit.com").build()

    fun getMinuteCandles(minute: Int) =
        getCandles("/v1/candles/minutes/$minute")

    fun getDayCandles() =
        getCandles("/v1/candles/days")

    fun getWeekCandles() =
        getCandles("/v1/candles/weeks")

    fun getMonthCandles() =
        getCandles("/v1/candles/months")

    fun getYearCandles() =
        getCandles("/v1/candles/years")

    private fun getCandles(path: String): List<CandleDto> {

        val result = restClient.get()
            .uri { b ->
                b.path(path)
                    .queryParam("market", "KRW-XRP")
                    .queryParam("count", 200)
                    .build()
            }
            .retrieve()
            .body(Array<CandleDto>::class.java)
            ?: error("캔들 데이터가 없습니다: $path")

        return result.toList().reversed()
    }
}
```

6. MultiTimeframeService - Java

```java
@Service
public class MultiTimeframeService {

    private final UpbitService upbitService;
    private final IndicatorService indicatorService;
    private final SignalService signalService;

    public MultiTimeframeService(
            UpbitService upbitService,
            IndicatorService indicatorService,
            SignalService signalService) {
        this.upbitService = upbitService;
        this.indicatorService = indicatorService;
        this.signalService = signalService;
    }

    private TimeframeAnalysisDto analyze(
            String timeframe,
            List<CandleDto> candles) {

        IndicatorDto indicators =
            indicatorService.calculate(candles);

        SignalDto signal =
            signalService.calculate(indicators);

        return new TimeframeAnalysisDto(
            timeframe, indicators, signal);
    }

    public List<TimeframeAnalysisDto> analyzeAll() {
        return List.of(
            analyze("15m",  upbitService.getMinuteCandles(15)),
            analyze("60m",  upbitService.getMinuteCandles(60)),
            analyze("240m", upbitService.getMinuteCandles(240)),
            analyze("1D",   upbitService.getDayCandles()),
            analyze("1W",   upbitService.getWeekCandles()),
            analyze("1M",   upbitService.getMonthCandles()),
            analyze("1Y",   upbitService.getYearCandles())
        );
    }
}
```

7. MultiTimeframeService - Kotlin

```kotlin
@Service
class MultiTimeframeService(
    private val upbitService: UpbitService,
    private val indicatorService: IndicatorService,
    private val signalService: SignalService
) {
    private fun analyze(
        timeframe: String,
        candles: List<CandleDto>
    ): TimeframeAnalysisDto {

        val indicators =
            indicatorService.calculate(candles)

        val signal =
            signalService.calculate(indicators)

        return TimeframeAnalysisDto(
            timeframe = timeframe,
            indicators = indicators,
            signal = signal
        )
    }

    fun analyzeAll(): List<TimeframeAnalysisDto> =
        listOf(
            analyze("15m",  upbitService.getMinuteCandles(15)),
            analyze("60m",  upbitService.getMinuteCandles(60)),
            analyze("240m", upbitService.getMinuteCandles(240)),
            analyze("1D",   upbitService.getDayCandles()),
            analyze("1W",   upbitService.getWeekCandles()),
            analyze("1M",   upbitService.getMonthCandles()),
            analyze("1Y",   upbitService.getYearCandles())
        )
}
```

8. 종합 결과 DTO

Java

```text
public record MultiTimeframeSignalDto(
    double bullishScore,
    double bearishScore,
    String signal,
    String strength
) {}
```

Kotlin

```kotlin
data class MultiTimeframeSignalDto(
    val bullishScore: Double,
    val bearishScore: Double,
    val signal: String,
    val strength: String
)
```

9. 가중 종합 점수 - Java

```java
public MultiTimeframeSignalDto calculateTotal(
        List<TimeframeAnalysisDto> list) {

    Map<String, Double> weights = Map.of(
        "15m",  0.05,
        "60m",  0.10,
        "240m", 0.15,
        "1D",   0.25,
        "1W",   0.20,
        "1M",   0.15,
        "1Y",   0.10
    );

    double bullish = 0;
    double bearish = 0;

    for (TimeframeAnalysisDto tf : list) {

        double weight =
            weights.getOrDefault(tf.timeframe(), 0.0);

        bullish +=
            tf.signal().bullishScore() * weight;

        bearish +=
            tf.signal().bearishScore() * weight;
    }

    String signal;
    if (bullish - bearish >= 15) {
        signal = "BULLISH";
    } else if (bearish - bullish >= 15) {
        signal = "BEARISH";
    } else {
        signal = "NEUTRAL";
    }

    double diff = Math.abs(bullish - bearish);

    String strength;
    if (diff >= 40) {
        strength = "STRONG";
    } else if (diff >= 20) {
        strength = "MEDIUM";
    } else {
        strength = "WEAK";
    }

    return new MultiTimeframeSignalDto(
        bullish, bearish, signal, strength);
}
```

10. 가중 종합 점수 - Kotlin

```kotlin
fun calculateTotal(
    list: List<TimeframeAnalysisDto>
): MultiTimeframeSignalDto {

    val weights = mapOf(
        "15m"  to 0.05,
        "60m"  to 0.10,
        "240m" to 0.15,
        "1D"   to 0.25,
        "1W"   to 0.20,
        "1M"   to 0.15,
        "1Y"   to 0.10
    )

    var bullish = 0.0
    var bearish = 0.0

    list.forEach { tf ->
        val weight = weights[tf.timeframe] ?: 0.0
        bullish += tf.signal.bullishScore * weight
        bearish += tf.signal.bearishScore * weight
    }

    val signal = when {
        bullish - bearish >= 15 -> "BULLISH"
        bearish - bullish >= 15 -> "BEARISH"
        else -> "NEUTRAL"
    }

    val diff = kotlin.math.abs(bullish - bearish)

    val strength = when {
        diff >= 40 -> "STRONG"
        diff >= 20 -> "MEDIUM"
        else -> "WEAK"
    }

    return MultiTimeframeSignalDto(
        bullishScore = bullish,
        bearishScore = bearish,
        signal = signal,
        strength = strength
    )
}
```

11. Gemma3에는 요약 결과만 전달

```text
예:
15m  상승=80 하락=20 BULLISH
60m  상승=90 하락=10 BULLISH
240m 상승=75 하락=25 BULLISH
1D   상승=70 하락=30 BULLISH
1W   상승=60 하락=40 BULLISH
1M   상승=45 하락=55 NEUTRAL/BEARISH
1Y   상승=55 하락=45 NEUTRAL

종합 상승점수=66
종합 하락점수=31
판정=BULLISH
강도=MEDIUM

→ 1,400개 원시 캔들을 LLM에 보내지 않고 위 요약 결과를 전달합니다.
```

12. Java - Gemma3 멀티프레임 설명

```java
public String analyze(
        List<TimeframeAnalysisDto> timeframes,
        MultiTimeframeSignalDto total) {

    StringBuilder data = new StringBuilder();

    for (TimeframeAnalysisDto tf : timeframes) {
        data.append(tf.timeframe())
            .append(" 상승=")
            .append(tf.signal().bullishScore())
            .append(" 하락=")
            .append(tf.signal().bearishScore())
            .append(" 판정=")
            .append(tf.signal().signal())
            .append("\n");
    }

    String prompt = """
        당신은 XRP 멀티 타임프레임 기술적 분석 설명 AI입니다.

        아래 결과는 Spring Boot가 직접 계산한 결과입니다.

        %s

        종합 상승점수: %.2f
        종합 하락점수: %.2f
        종합 판정: %s
        신호 강도: %s

        초단기 전망:
        단기 전망:
        중기 전망:
        장기 전망:
        초장기 전망:
        시간봉 간 일치 여부:
        상승 근거:
        하락 위험:
        종합 판단:

        점수를 확률이라고 표현하지 마세요.
        미래 가격을 확정적으로 예측하지 마세요.
        """.formatted(
            data,
            total.bullishScore(),
            total.bearishScore(),
            total.signal(),
            total.strength()
        );

    return chatModel.chat(prompt);
}
```

13. Kotlin - Gemma3 멀티프레임 설명

```kotlin
fun analyze(
    timeframes: List<TimeframeAnalysisDto>,
    total: MultiTimeframeSignalDto
): String {

    val data = timeframes.joinToString("\n") {
        "${it.timeframe} " +
        "상승=${it.signal.bullishScore} " +
        "하락=${it.signal.bearishScore} " +
        "판정=${it.signal.signal}"
    }

    val prompt = """
        당신은 XRP 멀티 타임프레임 기술적 분석 설명 AI입니다.

        아래 결과는 Spring Boot가 직접 계산한 결과입니다.

        $data

        종합 상승점수: ${total.bullishScore}
        종합 하락점수: ${total.bearishScore}
        종합 판정: ${total.signal}
        신호 강도: ${total.strength}

        초단기 전망:
        단기 전망:
        중기 전망:
        장기 전망:
        초장기 전망:
        시간봉 간 일치 여부:
        상승 근거:
        하락 위험:
        종합 판단:

        점수를 확률이라고 표현하지 마세요.
        미래 가격을 확정적으로 예측하지 마세요.
    """.trimIndent()

    return chatModel.chat(prompt)
}
```

14. Controller

Java

```java
@GetMapping("/multi-predict")
public Map<String, Object> multiPredict() {

    List<TimeframeAnalysisDto> timeframes =
        multiTimeframeService.analyzeAll();

    MultiTimeframeSignalDto total =
        multiTimeframeService.calculateTotal(timeframes);

    String aiAnalysis =
        aiService.analyze(timeframes, total);

    return Map.of(
        "timeframes", timeframes,
        "total", total,
        "aiAnalysis", aiAnalysis
    );
}
```

Kotlin

```kotlin
@GetMapping("/multi-predict")
fun multiPredict(): Map<String, Any> {

    val timeframes =
        multiTimeframeService.analyzeAll()

    val total =
        multiTimeframeService.calculateTotal(timeframes)

    val aiAnalysis =
        aiService.analyze(timeframes, total)

    return mapOf(
        "timeframes" to timeframes,
        "total" to total,
        "aiAnalysis" to aiAnalysis
    )
}
```

15. Java ↔ Kotlin 비교표

항목	Java	Kotlin
DTO	record	data class
목록	List.of()	listOf()
가중치	Map.of()	mapOf()
반복	for	forEach
문자열 조립	StringBuilder	joinToString()

항목	Java	Kotlin
DTO 값 접근	signal()	signal
분기	if / else	when
Spring 구조	동일	동일
LangChain4j/Gemma3	동일	동일
코드량	상대적으로 많음	상대적으로 간결

16. 실전 적용 시 중요한 점

연봉은 XRP의 데이터 개수가 적어 MA20, RSI14, MACD 같은 장기 지표가 충분히 안정적으로 계산되지 않을 수 있습니다. 따라서 연봉은 시가·고가·저가·종가, 전년도 대비 변화율, 장기 고점 대비 위치 같은 보조 정보로 쓰고 기술지표 점수는 15분·60분·240분·일봉·주봉·월봉 중심으로 구성하는 편이 좋습니다.

멀티 타임프레임 점수는 확률이 아닙니다. 가중치는 임의의 학습용 값이므로 실제 적용에서는 MariaDB에 예측시각, 프레임별 지표와 점수, 1시간·4시간·1일 후 실제 가격을 저장하여 백테스트로 조정하는 것이 다음 단계입니다.